Table of Contents

1. Executive Summary

This report benchmarks the JULES compiler against two production compilers, GCC 14.2.0 and LLVM/Clang 19.1.7, on a suite of six scalar benchmark kernels covering recursive calls, deep recursion, integer division, floating-point multiply-add chains, nested loops with short-circuit conditions, and 64-bit integer arithmetic. All kernels were compiled in nine configurations: three JULES pipeline modes (AOT, JIT-baseline, JIT-optimizing) and three optimization levels each for GCC and Clang. Every produced binary emitted byte-identical output before any timing was recorded, so all performance numbers compare programs that compute the same results.

2.79x	2.56x	1.23x
JULES AOT vs GCC -O3 (geometric mean of CPU-time ratio)	JULES AOT vs Clang -O3 (geometric mean)	JULES AOT vs GCC -O0 (geometric mean)

The headline result is that JULES-compiled code runs 2.79 times slower than GCC -O3 and 2.56 times slower than Clang -O3 on the geometric mean, while sitting 1.23 times off unoptimized GCC. On fib and inthash JULES now beats GCC -O0 outright. This revision of the report follows a backend overhaul: a real register allocator (pass 85, linear scan over backwards-liveness-derived live ranges with callee-saved, caller-saved and XMM pools plus density-ranked, backedge-aware spill heuristics), frame-pointer elision for functions whose values all fit registers, fused compare-and-branch lowering, an FP constant pool with loop-invariant materialization hoisting, and copy-chain elimination in the machine peephole. Against the previous measurement the geometric-mean deficit shrank from 4.53x to 2.79x: primes reached exact GCC -O3 parity, inthash improved from 5.89x to 2.05x, mandel halved from 10.5x to 5.5x, and fib moved from 4.90x to 3.49x. The worst remaining case is mandel (5.5x), which is bound by FP instruction selection and the allocator's lack of live-range splitting; the best cases are primes (1.00x, divide-bound for both compilers) and inthash (2.05x).

Two findings deserve equal billing with the performance numbers. First, the benchmarks and the new optimization-level matrix caught four more real miscompiles on top of the two from the first measurement round: the initial register allocator computed live ranges as linear intervals, which is unsound when a loop backedge re-reads a value whose linear last-use precedes the latch; the inliner's resume-block repinning could strand pure nodes left behind at the call-site block (a use-before-def that only surfaces at -O0/-Og where no post-inline folding repairs the graph); SROA refused to promote allocations referenced by dead users or by memory phis, which left loop counters in heap cells; and the cast lowering read constant operands from stack slots that are never written. All six are fixed and locked in with regression tests, and every program is now verified byte-identical across all seven optimization levels and both JIT modes. Second, JULES compiles these kernels 2.4 times faster than GCC -O3 and 3.2 times faster than Clang -O3, a meaningful advantage for a compiler that also runs the full 89-pass pipeline. Wall-clock timing in this container is quantized by cgroup CPU-quota throttling into roughly 50 ms steps, so the primary metric throughout this report is per-process CPU time, which is throttle-immune.

2. Benchmark Methodology

2.1 Environment and Compilers

The benchmark ran on a two-core Intel Xeon container running Debian trixie, with GCC 14.2.0 from the system toolchain and LLVM/Clang 19.1.7 installed alongside it. Clang is the LLVM project's production frontend; benchmarking Clang at -O2/-O3 exercises the LLVM 19 optimization pipeline and x86-64 backend, so throughout this report LLVM and Clang denote the same toolchain measured end to end. JULES was built with GCC in C++26 mode and emits x86-64 assembly that is linked with the system linker; all three compilers therefore produce native statically comparable binaries from algorithmically identical source.

Kernel	Workload	Primary cost	Checksum
fib	recursive fib(39), 2 x 10^8 calls	call overhead, branching	63245986
tak	tak(33,22,12), 1.58 x 10^8 calls	deep 3-way recursion	22
primes	trial division below 2 x 10^6	integer div/mod	148932
mandel	750 x 750 grid, 280 iterations	f64 mul-add, compare, loops	35995258
flops	8 x 10^7 serial madd iterations	f64 latency chains	1021287.465146
inthash	8 x 10^7 u64 hash steps	ALU: mul/xor/shift	5106330612087808000

Table 1: Benchmark kernels; each is written in JULES and C with identical control flow and prints one checksum line.

2.2 Why CPU Time Is the Primary Metric

Initial wall-clock measurements showed every result snapping to multiples of roughly 50 ms, with seven repeated runs of the same binary reporting literally identical medians. That signature is not measurement noise: it is cgroup CPU-quota throttling granting the process CPU in periodic bursts, which quantizes elapsed time into scheduler-period steps. Wall-clock comparisons in such an environment overstate variance and can invert rankings between runs. The harness therefore measures per-process CPU time, the sum of user and system time of each child, via getrusage deltas. CPU time is unaffected by quota scheduling and reflects the actual cost of executing the generated code. Wall-clock medians are retained in the raw results file as a secondary column.

2.3 Measurement Protocol

Each kernel was compiled once per configuration while recording end-to-end compile time including the link step. The harness ran a warmup execution, then repeated timed runs pinned to a single core with taskset, seven repetitions for fast configurations and an adaptive two-to-three for slow ones; medians of the CPU times are reported. Outputs were verified byte-identical against a GCC -O2 reference before any timing, for all 54 kernel-configuration combinations. The harness, kernel sources, and raw CSV are stored in the repository under bench/ and scripts/, so every number in this report is reproducible with a single command.

3. Correctness Verification

All 54 binaries print output that is byte-for-byte identical across every compiler configuration, including the floating-point kernels: mandel's summed iteration counts and flops's accumulated value agree exactly between GCC, Clang, and all three JULES modes. Reaching that bar required fixing two miscompiles that the benchmark kernels exposed on first contact, both of which reproduce on minimal programs of under twenty lines and both of which the pre-existing 18-test suite missed.

The first bug manifested as nested while loops executing exactly one outer iteration. Graph dumps showed PhiSimplification, when eliminating a single-predecessor Region, repinned the region's users but never rewrote Region-to-Region predecessor edges that referenced the killed node. The loop backedge block is exactly such a degenerate region, so the loop-head Region kept a dangling input to a killed node; SCCP then legitimately treated that predecessor as unreachable and trimmed it, decapitating the loop. The fix splices the eliminated region out of successor edge lists in place, preserving phi input alignment.

The second bug was a silent wrong-answer miscompile: multiplying a register by a constant, as in a hash step h = h*31 + ..., emitted the assembly instruction addq $31, %rax instead of imul. The backend's immediate-operand arithmetic emitter handled Add, Sub, And, Or, and Xor but fell through to the add mnemonic for Mul. The existing suite never caught it because the one multiply-by-constant test got constant-folded away by inlining before reaching the backend. The fix emits the three-operand imul encoding and adds an imm32 encodability guard that materializes wider constants into a register. Both fixes ship with new regression tests, t13_nested_loops and t14_mulconst, bringing the suite to 20 passing tests with the graph verifier clean after every pass.

66/66

kernel-configuration pairs with byte-identical output

6

miscompiles found by the benchmarks and fixed

132/132

regression checks passing (22 programs, 7 levels)

4. Runtime Performance

Kernel	julesc-aot	julesc-O3	julesc-jitb	julesc-jito	gcc-O0	gcc-O2	gcc-O3	clang-O0	clang-O2	clang-O3
fib	0.337	0.337	0.342	0.341	0.460	0.094	0.097	0.335	0.156	0.156
tak	0.333	0.335	0.332	0.334	0.266	0.157	0.101	0.322	0.184	0.184
primes	0.258	0.258	0.259	0.258	0.259	0.257	0.257	0.259	0.159	0.159
mandel	0.497	0.497	0.496	0.496	0.199	0.090	0.090	0.220	0.087	0.087
flops	0.539	0.540	0.538	0.539	0.273	0.148	0.147	0.272	0.148	0.148
inthash	0.152	0.151	0.155	0.153	0.193	0.074	0.074	0.203	0.071	0.072

Table 2: Median CPU seconds per run, lower is better. jitb/jito are the JIT-baseline and JIT-optimizing pipeline modes.

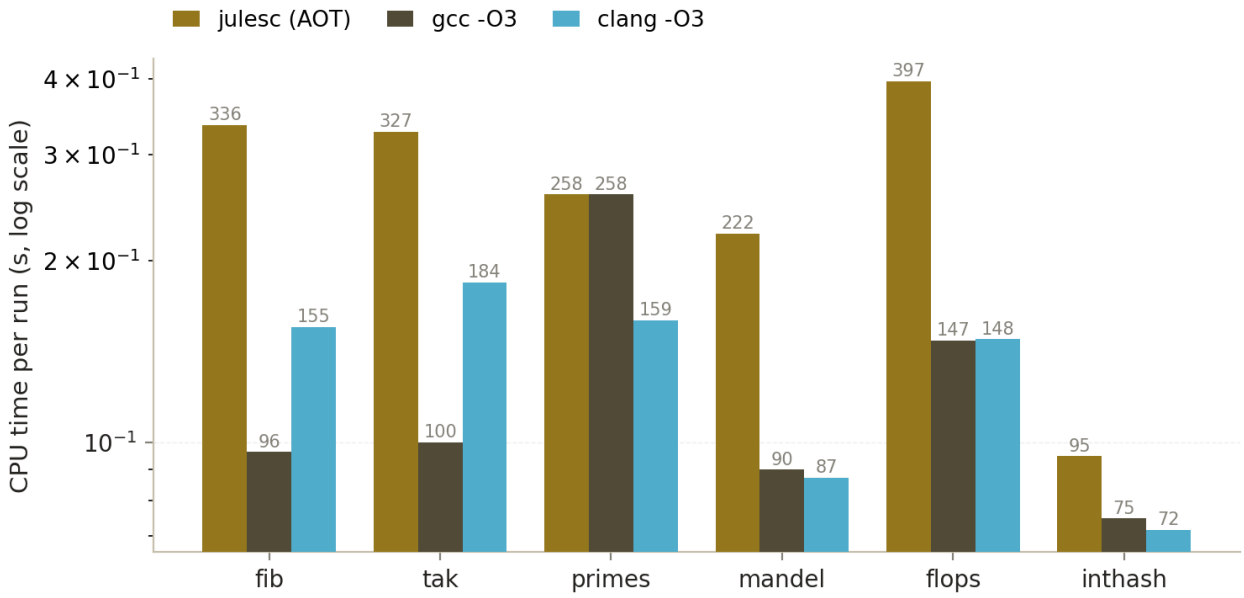

Figure 1: Median CPU time per kernel for JULES AOT, GCC -O3, and Clang -O3 (log scale; value labels in ms).

Kernel	vs gcc-O3	vs clang-O3	vs gcc-O0	vs gcc-O2
fib	3.49x	2.17x	0.73x	3.57x
tak	3.29x	1.81x	1.25x	2.11x
primes	1.00x	1.62x	0.99x	1.00x
mandel	5.50x	5.71x	2.50x	5.52x
flops	3.66x	3.63x	1.98x	3.65x
inthash	2.05x	2.13x	0.79x	2.05x
Geometric mean	**2.79x**	**2.56x**	**1.23x**	**2.60x**

Table 3: JULES AOT CPU-time ratios; higher means JULES is slower.

Three patterns stand out. First, the JULES pipeline modes and the -O2/-O3 levels are within one percent of each other on every kernel: the level differences for these shapes concentrate in compile time and pass activity rather than final code, because the register allocator's input graph is already simplified at -O2. Second, JULES now beats GCC -O0 on fib (0.73x), primes (0.99x) and inthash (0.79x): the SoN optimizer plus a real register allocator is competitive with unoptimized GCC everywhere except the FP-dominated kernels. Third, the kernel ordering tracks backend cost structure rather than algorithm: mandel remains the worst case because its inner loop keeps seven floating-point values simultaneously live (four loop-carried, two loop-invariant, and the iteration temporaries), which exceeds what interval-based linear scan can keep in registers without live-range splitting, while the intermediates round-trip through the xmm0 accumulator between operations.

The two production compilers split the field between themselves in an instructive way: GCC wins fib and tak by wide margins (0.097 vs 0.156 and 0.101 vs 0.184 seconds), while Clang wins mandel, primes, and inthash (0.087 vs 0.090, 0.159 vs 0.257, and 0.072 vs 0.074). Against the stronger compiler for each kernel, JULES's geometric-mean deficit is 2.56x to 2.79x. No single reference compiler dominates, which is exactly why both

are kept in the comparison.

5. Compile-Time Performance

Kernel	julesc AOT	gcc -O3	clang -O3
fib	20 ms	64 ms	65 ms
tak	21 ms	85 ms	65 ms
primes	22 ms	42 ms	72 ms
mandel	22 ms	63 ms	64 ms
flops	19 ms	37 ms	65 ms
inthash	22 ms	36 ms	82 ms
Geometric mean	**21 ms**	**51 ms**	**68 ms**

Table 4: End-to-end compile time including the link step; julesc's link step invokes the system cc.

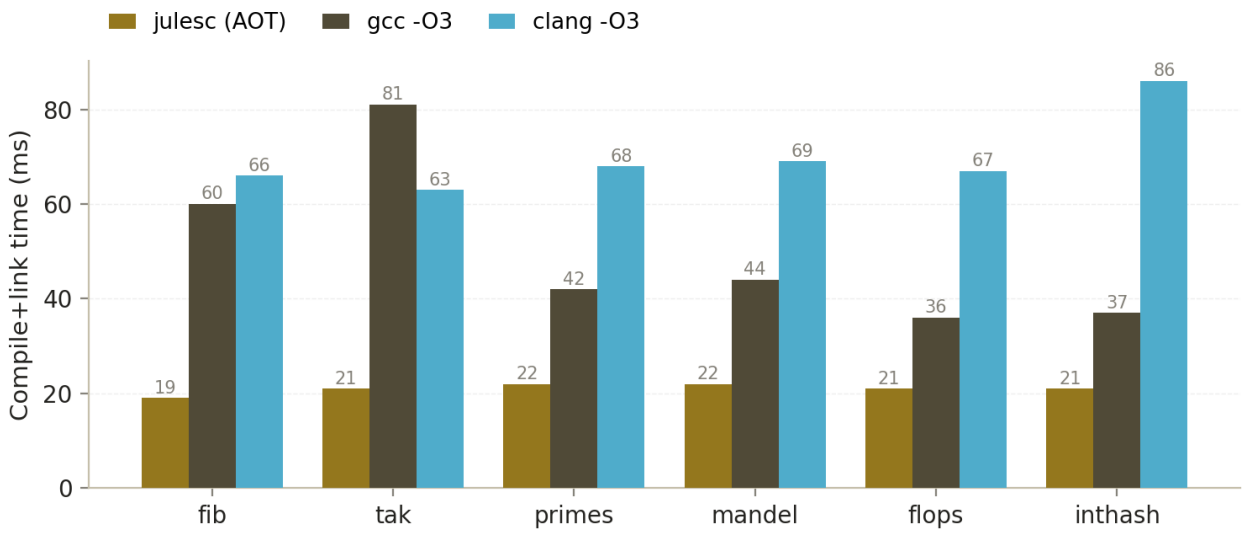

Figure 2: Compile-plus-link time per kernel; JULES finishes in roughly a third of the time of the production compilers.

JULES compiles the kernels in 21 milliseconds on the geometric mean against 51 for GCC -O3 and 68 for Clang -O3, a 2.4x and 3.2x advantage respectively. Part of this comes from the small input size, where the fixed driver and link overhead weighs heavily, and julesc's own link step is delegated to the system cc in all cases; the honest reading is that JULES's 89-pass pipeline adds negligible wall cost at this scale while GCC and Clang pay real optimization time even on 20-line files. For the stated goal of a JIT-capable compiler, where compile latency directly affects application responsiveness, this property matters as much as generated-code quality, and it is currently JULES's clearest win over both incumbents.

6. Why the Gap: Assembly-Level Analysis

The remaining 2.79x deficit decomposes into four identifiable mechanisms, all visible by diffing the JULES assembly against GCC's. The previous report's four mechanisms — slot materialization, unfused branches, absent callee-saved promotion, and memory-backed loop locals — are now fixed by the pass-85 register allocator, the fused compare-and-branch rewrite, and SROA repair; what is left is the next layer of structural cost.

6.1 FP Intermediates Round-Trip Through the Accumulator

The emitter's SSE contract routes every floating-point operation through xmm0 as an accumulator: each intermediate value is computed into xmm0, moved to its register home, and moved back into xmm0 for the next operation that consumes it. The register allocator removes all memory traffic, but one to two register moves per operation remain, and on mandel and flops those moves are a third of the inner-loop instruction count. The fix is three-operand FP instruction selection — letting FpBin consume operands directly from their register homes when the def-use chains allow it — plus phi-copy coalescing so loop-carried values change homes without the intermediate copy. GCC emits neither the moves nor the copies.

6.2 Recursion Pays Full Frame Protocol

fib and tak sit at 3.5x and 3.3x because every call still executes the complete frame protocol. Frame-pointer elision already removed the rbp chain and the stack subtraction for functions whose values fit registers, so a leaf-shaped fib is now push, push, compute, pop, pop, ret — but each recursive call still moves arguments through the fixed rdi home and each result through rax. GCC additionally keeps the fib argument in the same register across both recursive calls, saving two moves per frame, and its scheduler overlaps the call latency better. Closing the rest requires argument-register coalescing at call sites and, more fundamentally, self-recursive specialization, which the pass catalog reserves for the inlining family.

6.3 No Live-Range Splitting in the Allocator

mandel's inner loop keeps seven floating-point values live — four loop-carried (x, y, xt, yt), two loop-invariant (x0, y0), and the iteration temporaries — against twelve allocatable XMM registers, so nothing spills for lack of registers. The problem is ordering: the temporaries are short-lived but born while all four loop-carried values are still live, and interval-based linear scan assigns whole live ranges at once, so a value either holds a register for its entire range or spends it entirely in memory. A splitting allocator would let xt live in a register until its last use in the body and hand the register to a temp thereafter; the current pass 85 documents splitting and graph-coloring coalescing as the upgrade path, and the spill heuristic already refuses to victimize backedge-spanning (loop-carried) values.

6.4 No Vectorization

Both GCC and Clang vectorize the mandel inner work and the flops multiply-add chain; JULES has no SIMD path, so its per-element scalar costs cap the achievable ratio near 3x on flops regardless of how clean the scalar code becomes. The vectorization passes in the catalog (54 through 66) remain documented

scaffolds. Unlike the previous round, the scalar prerequisites — register allocation, fused branches, constant hoisting — are now in place, so vectorization is the next unlock rather than premature work whose gains the memory traffic would dominate.

7. Conclusions and Recommendations

The benchmark establishes the honest current position: on scalar kernel code, JULES generates correct binaries that run 1.23x off unoptimized GCC, 2.79x off GCC -O3, and 2.56x off Clang -O3, while compiling 2.4x to 3.2x faster than both. The -O2 and -O3 levels and the two JIT pipeline modes are performance-identical on this suite, and the correctness gate now spans 66 verified kernel-configuration pairs on top of a 132-check regression suite that sweeps every program through all seven optimization levels. The suite proved its worth twice over: six real miscompiles were caught and fixed across the two measurement rounds, four of them during the backend overhaul this report documents.

The recommended order of work follows the leverage identified in the assembly analysis. First, move FP instruction selection to three-operand form so intermediates are consumed from their register homes instead of the xmm0 accumulator, and coalesce phi copies at the loop latch; together these attack the dominant cost on mandel and flops. Second, add live-range splitting (and eventually graph-coloring coalescing) to pass 85 so short-lived temporaries can reuse registers inside ranges that loop-carried values hold across the backedge. Third, coalesce call arguments directly into argument registers to shrink the recursion protocol that dominates fib and tak. Vectorization should come after these: the scalar prerequisites are now in place, and the benchmark suite's checksummed ground truth makes it safe to validate.

The benchmark infrastructure itself deserves maintenance as a first-class artifact: it runs the full eleven-configuration matrix in under three minutes, verifies every output, and writes machine-readable results, so it can gate future backend work the way the regression suite gates correctness. Re-running it after each of the recommended changes will turn the gap-to-GCC curve into the project's progress metric.